TP - Gestion des livraisons

Algorithmie comparative, M2 Architecture et développement logiciel
Dépôt : https://github.com/BabyNewCode/tp-gestion-livraisons

1. Le problème et le choix du langage

Le sujet se découpe en deux morceaux de nature assez différente. D'un côté, il faut gérer une collection de colis qui peut monter à 100 000 éléments : ajouter, retrouver un colis par son identifiant, afficher les colis triés. De l'autre, il faut remplir un camion de capacité limitée en maximisant la valeur transportée. Ce second morceau est le problème du sac à dos 0/1, connu pour être NP-difficile, et c'est là que se jouent la plupart des choix.

Nous avons fait le choix de Python. Le TP porte sur les algorithmes et leurs complexités, pas sur la gestion fine de la mémoire, et Python permet d'écrire, de tester (`unittest`) et de mesurer (`time.perf_counter`) sans rien installer. Le prix à payer est la vitesse brute : une boucle Python est nettement plus lente que la même boucle en C. Les mesures le montrent, mais les complexités, elles, ne dépendent pas du langage. Aucune bibliothèque externe n'a été utilisée.

Les choix présentés ci-dessous sont dans l'ordre du sujet : stockage et recherche, tri, chargement du camion, puis prise en compte de la priorité. La dernière partie analyse le comportement quand le nombre de colis augmente.

2. Choix n°1 : stocker les colis dans une table de hachage

L'opération la plus fréquente pour l'entreprise est la recherche d'un colis par son identifiant. Avec une simple liste, chaque recherche oblige à parcourir les colis un par un, donc O(n). À 100 000 colis, ça devient sensible si on recherche souvent. Nous avons donc stocké les colis dans un dictionnaire Python (`id → Colis`), qui est une table de hachage : l'identifiant est haché pour tomber directement sur la bonne case.

Un détail nous a simplifié la vie : depuis Python 3.7, le dictionnaire conserve l'ordre d'insertion. On peut donc afficher les colis « dans l'ordre d'arrivée » sans maintenir une liste à côté. Le dictionnaire permet aussi de refuser un identifiant en double en O(1) au moment de l'ajout.

Problème	Ajouter et retrouver rapidement un colis par identifiant parmi jusqu'à 100 000 colis.
Solution	Dictionnaire `id → Colis` (table de hachage).
Temps	Ajout O(1) en moyenne, recherche O(1) en moyenne. Pire cas O(n) en cas de collisions massives, très improbable avec des entiers.
Espace	O(n). La table garde de la place libre pour limiter les collisions, donc un peu plus que n cases.
Alternative	Liste non triée avec recherche linéaire en O(n), ou liste triée par id avec recherche dichotomique en O(log n), mais l'insertion coûte alors O(n) à cause du décalage.
Justification	On recherche et on ajoute bien plus souvent qu'on ne parcourt tout dans l'ordre des id. La table de hachage est la seule option en O(1) pour les deux. Mesuré : 0,1 µs par recherche, que n vaille 100 ou 100 000.

3. Choix n°2 : un tri fusion écrit à la main

Le sujet demande d'afficher les colis selon leur poids, leur valeur ou leur priorité. La solution la plus simple aurait été d'appeler `sorted()`. Or le sujet précise qu'appeler une fonction native sans pouvoir en expliquer le fonctionnement ne suffit pas. Nous avons donc codé un tri fusion, dont `sorted()` (Timsort) est d'ailleurs un cousin.

Le principe : on coupe la liste en deux, on trie chaque moitié récursivement, puis on fusionne les deux moitiés triées en comparant leurs têtes. Il y a log n niveaux de découpe et chaque niveau fusionne n éléments au total, d'où O(n log n). Contrairement au tri rapide, il n'y a pas de pire cas en $O(n^2)$. Le tri est aussi stable : en cas d'égalité, on prend toujours l'élément de gauche. Deux colis de même priorité gardent donc leur ordre d'arrivée, ce qui est le comportement qu'on attend d'un affichage.

Nous avons ajouté trois petites optimisations, sans changer la complexité. Le tampon de fusion est alloué une seule fois au lieu d'être recréé à chaque appel. Les clés sont calculées une seule fois au départ (n appels au lieu de n log n). Enfin, si les deux moitiés sont déjà dans le bon ordre, la fusion est sautée. Sur une liste déjà triée, le tri devient alors presque linéaire : 41 ms au lieu de 353 ms à 100 000 colis.

Problème	Afficher les colis triés selon un critère au choix (poids, valeur, priorité), dans un sens ou dans l'autre.
Solution	Tri fusion récursif, stable, avec un tampon unique et des clés précalculées.
Temps	O(n log n) dans tous les cas. Proche de O(n) si la liste est déjà triée.
Espace	O(n) pour le tampon et les clés, plus O(log n) pour la pile de récursion.
Alternative	Tri rapide (en place, mais $O(n^2)$ au pire et non stable), tri par tas (O(n log n), en place, non stable). Le tri par comptage serait en O(n + k) pour la priorité, qui ne prend que 5 valeurs.
Justification	Garantie O(n log n) sans pire cas, stabilité utile pour l'affichage, et un algorithme facile à expliquer et à vérifier. Testé contre `sorted()` sur des données aléatoires.

Le tri par comptage mériterait d'être retenu pour la priorité seule : avec 5 valeurs possibles, il est linéaire. Nous ne l'avons pas fait pour garder un seul algorithme pour les trois critères, puisque le poids et la valeur n'ont pas un domaine aussi réduit.

4. Choix n°3 : le chargement du camion en programmation dynamique

On dispose de n colis (poids p_i, valeur v_i) et d'une capacité W. On cherche le sous-ensemble de valeur maximale dont le poids total ne dépasse pas W, chaque colis étant pris en entier ou pas du tout. Le sujet insiste : une combinaison qui respecte la capacité ne suffit pas, il faut la meilleure valeur possible.

Pourquoi cette stratégie ?

À première vue, une approche gloutonne semble naturelle : trier les colis par rapport valeur/poids et les prendre tant qu'ils rentrent. Mais elle n'est pas optimale pour le sac à dos 0/1, et l'exemple du sujet suffit à le montrer. Avec une capacité de 9 kg, le glouton prend d'abord B (ratio 4,17), puis il ne reste que 3 kg, ce qui ne suffit pour aucun autre colis : valeur 25. Or C + D pèsent exactement 9 kg pour une valeur de 32. Le glouton a choisi le meilleur colis isolément, mais il a laissé un trou que d'autres colis auraient mieux rempli.

La programmation dynamique évite ce piège. On note `meilleur[w]` la meilleure valeur atteignable avec une capacité w en n'utilisant que les colis déjà examinés. Pour chaque nouveau colis (p, v), on met à jour :

```
meilleur[w] = max( meilleur[w], meilleur[w – p] + v )   pour w allant de W à p
```

Autrement dit, pour chaque capacité, on compare « ne pas prendre le colis » et « le prendre en lui réservant p kg ». Parcourir w de W vers p, à l'envers, garantit que chaque colis n'est compté qu'une fois. Dans l'autre sens, on autoriserait sans le vouloir à prendre le même colis plusieurs fois.

Connaître la meilleure valeur ne suffit pas, il faut savoir quels colis charger. Pour chaque colis, on garde donc une ligne de décisions (« a-t-il amélioré `meilleur[w]` ? »), puis on remonte depuis w = W : si le dernier colis a été pris à cette capacité, on l'ajoute au chargement et on retire son poids, puis on passe au colis précédent.

La solution est-elle toujours optimale ?

Oui, pour des poids entiers. La récurrence explore implicitement toutes les combinaisons, sans les énumérer une par une, puisque chaque sous-problème (colis 1..i, capacité w) n'est résolu qu'une fois. Pour le vérifier en pratique, nous avons codé une force brute qui teste les 2^n sous-ensembles et nous l'avons comparée à la programmation dynamique sur 150 instances aléatoires de 12 colis : les valeurs sont identiques à chaque fois. La contrepartie, c'est que les poids doivent être entiers, ou ramenés à une unité entière (le kilo ici).

Problème	Choisir les colis à charger pour maximiser la valeur sans dépasser la capacité W (sac à dos 0/1).
Solution	Programmation dynamique sur la capacité, tableau 1D parcouru à l'envers, reconstruction par une table de décisions.
Temps	O(n·W). C'est pseudo-polynomial : polynomial en la valeur de W, pas en sa taille en bits.
Espace	O(W) pour les valeurs, plus n·(W+1) octets pour les décisions (un `bytearray` par colis, 1 octet par case). Une liste d'entiers Python aurait coûté environ 8 fois plus.
Alternative	Glouton par ratio : O(n log n), mais pas optimal. Force brute : $O(2^n)$, inutilisable au-delà d'une vingtaine de colis. Séparation et évaluation (branch and bound) : optimale, souvent rapide, mais sans garantie sur le pire cas.
Justification	Le sujet exige la meilleure valeur, ce qui écarte le glouton. Pour des capacités de camion réalistes (quelques centaines à quelques milliers de kg), O(n·W) reste raisonnable, et la méthode se prête bien à la reconstruction de la solution.

Aurait-on pu faire autrement ?

Oui. Nous avons gardé le glouton dans le programme comme option « rapide », parce qu'il a un intérêt réel. Sur nos données aléatoires, il atteint 99,9 à 100 % de l'optimum dès qu'on charge quelques centaines de colis. Ce n'est pas un hasard : l'écart entre le glouton et l'optimum est borné par la valeur d'un seul colis, qui devient négligeable quand on en charge beaucoup. Il reste que l'écart n'est pas nul (99,86 % à 1 000 colis) et que, sur des petits cas comme celui du sujet, il peut être important. La programmation dynamique reste donc la méthode par défaut.

5. Choix n°4 : une règle pour la priorité

L'entreprise veut favoriser les colis urgents. Le sujet n'impose pas de formule. Nous avons considéré que 5 est la priorité la plus haute, puis choisi de ne pas toucher à l'algorithme mais à ce qu'il optimise : au lieu de maximiser la valeur, la programmation dynamique maximise un score.

```
score = valeur × (1 + α × (priorité − 1) / 4)
```

Un colis de priorité 1 compte pour sa valeur, un colis de priorité 5 pour sa valeur multipliée par (1 + α). Le paramètre α est réglable par l'utilisateur : à 0, on retrouve le chargement sans priorité ; à 1, un colis

très urgent « compte double ». Il nous a semblé plus honnête de laisser ce curseur à l'entreprise que de fixer arbitrairement l'importance de l'urgence.

Pourquoi un facteur multiplicatif plutôt qu'un bonus fixe ajouté à la valeur ? Avec un bonus fixe, un colis urgent de valeur quasi nulle deviendrait plus intéressant qu'un colis moins urgent mais précieux. Le multiplicateur amplifie la valeur sans l'écraser. Quant au poids, on ne l'a pas mis dans le score : il reste une contrainte dure (la capacité), et c'est déjà à la programmation dynamique d'arbitrer entre ce qu'un colis rapporte et la place qu'il prend.

Sur l'exemple du sujet avec 13 kg, le chargement sans priorité est B + C (valeur 43). Avec $\alpha = 1$, il devient A + C : A, de priorité 5, passe devant B, de priorité 1, alors que sa valeur est plus faible. Sur 1 000 colis aléatoires et une capacité de 300 kg, l'effet est progressif :

α	Valeur transportée	Priorité moyenne chargée
0	5 499	3,17
1	5 366	3,55
3	5 134	3,83

On sacrifie un peu de valeur (−2,4 % avec $\alpha = 1$) pour charger des colis nettement plus urgents.

Problème	Faire influencer la sélection par la priorité, en plus du poids et de la valeur.
Solution	Optimiser un score multiplicatif paramétré par α avec la même programmation dynamique.
Temps	Inchangé : O(n·W). Le calcul du score est en O(1) par colis.
Espace	Inchangé.
Alternative	Bonus additif ; ou règle stricte : charger d'abord tous les colis de priorité 5 qui rentrent, puis optimiser la valeur sur la place restante.
Justification	La règle stricte peut gâcher beaucoup de place avec des colis urgents mais lourds et peu utiles. Le score garde l'optimalité de la PD, se règle avec un seul paramètre et n'a demandé aucune réécriture d'algorithme.

6. Comportement quand n augmente

Toutes les mesures ont été faites sur des données aléatoires reproductibles (seed 42 ; poids de 1 à 50 kg ; valeurs de 1 à 100). Le script est `src/benchmark.py` et les résultats complets sont dans `resultats/benchmark.md`. Nous avons retenu trois opérations dont le coût dépend de la quantité de données.

Recherche d'un colis

n	Recherche dans le dictionnaire	Recherche linéaire	Rapport
100	0,1 µs	1,7 µs	×21
1 000	0,1 µs	13,6 µs	×151
10 000	0,1 µs	148 µs	×1 656
100 000	0,1 µs	1,8 ms	×19 773

La complexité est O(1) en moyenne. Le temps reste constant quand n est multiplié par 1 000, alors que la recherche linéaire, en O(n), suit n presque exactement. Le choix reste adapté sans réserve, et nous ne

voyons pas d'amélioration utile. La seule limite serait la mémoire, à des tailles très au-delà de celles du sujet.

Tri des colis

n	Tri fusion (maison)	sorted() (Timsort, en C)	Tri fusion sur données déjà triées
100	0,11 ms	0,008 ms	0,03 ms
1 000	2,0 ms	0,13 ms	0,25 ms
10 000	26 ms	1,8 ms	3,1 ms
100 000	353 ms	20 ms	41 ms

La complexité est O(n log n) : quand n est multiplié par 10, le temps est multiplié par 13 environ, un peu plus que 10, ce qui correspond au facteur log n. Notre tri est environ 15 fois plus lent que `sorted()`, pour une complexité identique : l'écart vient de l'interpréteur Python et non de l'algorithme. Un tiers de seconde à 100 000 colis reste acceptable pour un affichage. En revanche, retrier à chaque affichage est du gaspillage si la liste n'a pas changé. L'amélioration que nous proposons est de garder en cache le résultat de chaque tri et de l'invalider lors d'un ajout. Une autre piste serait de maintenir directement des listes triées, avec une insertion en O(n) par recherche dichotomique et décalage.

Chargement du camion

C'est l'opération la plus coûteuse, en O(n·W). Nous l'avons donc mesurée en faisant varier n puis W.

n (W = 500)	PD	Glouton	Glouton / optimum	Mémoire des décisions
100	5,5 ms	0,2 ms	100 %	0,1 Mo
1 000	37 ms	2,5 ms	99,86 %	0,5 Mo
10 000	354 ms	29 ms	100 %	5 Mo
100 000	3,59 s	406 ms	100 %	50 Mo

W (n = 1 000)	PD	Glouton	Glouton / optimum
100	5,8 ms	2,1 ms	100 %
1 000	85 ms	1,8 ms	99,97 %
10 000	1,10 s	2,8 ms	100 %
50 000	6,31 s	2,0 ms	100 %

Le temps est bien linéaire en n à W fixé, et linéaire en W à n fixé. Pour une capacité de camion réaliste, la programmation dynamique reste utilisable jusqu'à 100 000 colis : environ 3,6 s et 50 Mo. C'est la capacité qui pose problème quand elle grandit, beaucoup plus que le nombre de colis. Le dernier cas est parlant : avec 50 000 kg de capacité, tous les colis rentrent (environ 25 000 kg au total), et la programmation dynamique passe pourtant 6 s à calculer un résultat évident.

Le choix reste adapté au cas d'usage, mais il a des limites, d'où trois améliorations possibles. La plus simple consiste à borner W par le poids total des colis candidats : on éviterait le cas absurde ci-dessus, et si tout rentre, la réponse est immédiate. Ensuite, compter les poids par tranches (par exemple de 5 kg) divise W, donc le temps et la mémoire, au prix d'une petite approximation. Enfin, au-delà d'un seuil de n·W, le glouton devient le meilleur compromis, puisque son écart à l'optimum est faible sur de grands volumes. Le menu applique déjà une forme de cette dernière idée : il prévient l'utilisateur avant

de lancer une programmation dynamique de plus de 50 millions de cases, ce qui lui laisse la possibilité d'annuler et de choisir le glouton.

7. Ce que nous retenons

Les deux premières opérations, recherche et tri, se règlent avec des structures et des algorithmes classiques dont le coût reste maîtrisé jusqu'à 100 000 colis. Le chargement du camion est d'une autre nature : la meilleure solution exacte coûte $O(n \cdot W)$. C'est acceptable pour des capacités réalistes, mais il faut savoir quand passer à une approximation. Le point qui nous a le plus surpris est la qualité du glouton sur de grands volumes : il n'est pas optimal en théorie, et l'exemple du sujet le montre, mais il s'en approche à quelques dixièmes de pourcent près dès qu'on charge des centaines de colis. Garder les deux méthodes et laisser le choix à l'utilisateur nous paraît être la réponse la plus raisonnable.